Universität Trier

Computational Linguistics and Digital Humanities

Department II

Term Paper

Machine Learning for Natural Language Understanding

Dataset Structure Matters:

A Dataset-Centric Structural Analysis and Empirical Study of Instruction-Fine Tuning Corpora

submitted for the degree of

Master of Science

Natural Language Processing

Winter Semester 2025/2026

Presented by

Yash Gavade - 1757209

s4yagava@uni-trier.de

Submission Date: March 24, 2026

Supervisors: Prof. Dr. Achim Rettinger
Asst. Prof. Raghvi Baloni

Contents

1 Introduction **1**

2 Background and Related Work **2**

2.1 Large Language Model Training Pipeline 2

2.2 Instruction Fine-Tuning Datasets 2

2.3 Dataset-Centric Perspective 3

3 Dataset Selection and Description **3**

3.1 Alpaca Dataset 3

3.2 Dolly Dataset 3

3.3 OpenAssistant Dataset 3

3.4 Rationale for Dataset Selection 3

4 Methodology **4**

4.1 Data Preprocessing and Standardization 4

4.2 Length and Distributional Metrics 4

4.3 Lexical Diversity and Entropy 4

4.4 Redundancy and Template Analysis 4

4.5 Clustering-Based Semantic Dispersion 5

4.6 Extended Structural Metrics 5

5 Structural Analysis Results **5**

5.1 Dataset Size and Length Distributions 5

5.2 Lexical Diversity and Entropy 6

5.3 Redundancy and Template Regularity 6

5.4 Clustering-Based Semantic Dispersion 6

6 Fine-Tuned Model Evaluation **7**

6.1 Experimental Setup 7

6.2 Evaluation Prompt Set 7

6.3 Manual Evaluation Framework 7

6.4 Manual Evaluation Results 8

6.5 Constraint-Following Evaluation 8

6.6 Qualitative Comparison of Model Outputs 8

7 Discussion **9**

8 Limitations 9

9 Conclusion 10

References 11

A Appendix: Dataset Processing and Structural Analysis Pipeline 12

A.1 Experimental Workflow . . . 12

A.2 Dataset Processing Pipeline . . . 12

A.3 Structural Analysis Implementation . . . 13

A.4 Fine-Tuning Setup . . . 13

A.5 Evaluation Workflow and Scripts . . . 14

A.6 Additional Analysis Results . . . 15

A.6.1 Clustering-Based Semantic Dispersion . . . 15

A.6.2 Cross-Dataset Similarity . . . 16

A.6.3 Example Evaluation Prompts . . . 17

Declaration of Authenticity 18

List of Figures

1 Dataset-centric workflow including preprocessing, structural analysis, fine-tuning, and evaluation. 4

2 Instruction length distributions across datasets. 5

3 Output length distributions (histograms) across datasets. 5

4 Vocabulary size and entropy comparison across datasets. 6

5 Clustering-based semantic dispersion across datasets. 6

6 Manual evaluation scores across models 8

7 Constraint-following performance 8

8 Dataset-centric research framework illustrating preprocessing, structural analysis, model fine-tuning, and evaluation stages. 12

9 Dataset preprocessing pipeline used to standardize instruction-tuning datasets. 12

10 Fine-tuning workflow illustrating LoRA-based adaptation of TinyLLaMA models 14

11 Training loss curve across fine-tuning steps 14

12 Evaluation workflow including response generation, scoring, and analysis 14

13 Final evaluation summary comparing model performance across metrics 15

14 Category-wise average response length across models 15

15 Category-wise response entropy across models 15

16 Clustering-based semantic dispersion across datasets 16

17 Category-wise manual score heatmap 16

18 Overall mean manual score comparison 16

19 Repeated line rate across models 17

20 Repeated trigram ratio across models 17

List of Tables

1 Dataset construction paradigms . . . 3

2 Summary statistics of instruction-tuning datasets . . . 5

3 Lexical diversity and entropy across datasets . . . 6

4 Manual evaluation criteria used for assessing model outputs . . . 7

5 Manual evaluation scores across models (IF: Instruction Following, Corr: Correctness, Clar: Clarity, Comp: Completeness) . . . 8

6 Constraint-following success rates . . . 8

7 Training configuration used for all fine-tuning experiments . . . 13

Dataset Structure Matters:

A Dataset-Centric Structural Analysis and Empirical Study of Instruction-Tuning Corpora

Yash Gavade
Matr.-Nr.: 1757209
s4yagava@uni-trier.de
Machine Learning for Natural Language Understanding
Winter Semester 2025/2026
Schriftliche Ausarbeitung
P1240M0008e — Prüfung
Universität Trier, Trier, Germany

Abstract

Instruction fine-tuning is a crucial part of aligning large language models (LLMs) to human tasks. Most studies have focused on model development and performance, but there has been little investigation into instruction dataset structure. However, instruction datasets are developed in varying ways, such as synthetic, human-written, or conversational, which might affect model learning and behavior.

This study presents a dataset-centric analysis of three widely used instruction datasets: Alpaca, Dolly, and OpenAssistant (OASST1). First, all datasets are standardized into a unified format, and their structural properties are analyzed using metrics such as length distribution, lexical diversity, entropy, redundancy, and semantic clustering. In addition, instruction types and cross-dataset similarity are examined.

A controlled experiment is then conducted by fine-tuning TinyLLaMA using LoRA on each dataset. The fine-tuned models are evaluated using a shared set of prompts and manual scoring.

The results show that there are considerable structural variations between instruction datasets. For instance, synthetic datasets are more repetitive and more structured compared to human-written datasets, which are more diverse and varied in terms of content. Overall, this study demonstrates that instruction dataset structure plays a crucial part in instruction fine-tuning, just as model development does.

Keywords: Instruction Fine-Tuning, Large Language Models (LLMs), Dataset Structure, Alpaca Dataset, Dolly Dataset, OpenAssistant, TinyLLaMA, LoRA, Dataset-Centric Analysis, Semantic Clustering

Github: https://github.com/Yash-Gavade/Dataset-Structure-Matters

1 Introduction

Significant advancements in the field of natural language processing have been achieved using large language models (LLMs) in recent times, especially owing to the capability of these models to perform a wide range of duties such as summarization, translation, question answering, and code generation using a single model [Brown et al., 2020, Vaswani et al., 2017, Devlin et al., 2019, Touvron et al., 2023, OpenAI, 2023].

Instruction fine-tuning, among the various alignment approaches, has become an important aspect in the development of language models that are capable of responding effectively to natural language instructions [Wei et al., 2022, Ouyang et al., 2022, Wang et al., 2022]. In the process of instruction fine-tuning, the models are trained on examples with an instruction, optional input, and output, allowing the models to generate effective responses. Although instruction fine-tuning is popular, most studies have focused mainly on model architecture, fine-tuning methods, and evaluation methods [Ouyang et al., 2022, Longpre et al., 2023a, Ding et al., 2023].

However, instruction datasets have not received much attention, especially with regard to their structural characteristics [Liu et al., 2024]. Instruction datasets have different construction methodologies, such as synthetic data, human annotation, and conversational data, which may affect how instruction models fine-tune and behave during fine-tuning.

This study takes a dataset-centric approach to explore instruction datasets, specifically instruction datasets such as Alpaca, Dolly, and OpenAssistant (OASST1) [Taori et al., 2023, Databricks, 2023, OpenAssistant, 2023], which are popular instruction datasets with different construction paradigms, such as synthetic data, human annotation, and conversational data, respectively.

To do this, this study uses a two-stage approach. First, instruction datasets are standardized to a uniform format, then their structural characteristics, such as length, lexical variety, entropy, redundancy, and semantic clustering, are computed using various metrics [Shannon, 1948, Salton and Buckley, 1988, MacQueen, 1967]. Second, a controlled experiment is conducted, where instruction fine-tuning is carried out using the LoRA method, specifically using the TinyLLaMA model [Hu et al., 2022, Zeng et al., 2024], then tested using shared prompts, scoring, and constraint following.

This study aims to answer the following research questions:

- **RQ1:** In what ways do instruction datasets vary in terms of structural properties such as length distribution, lexical diversity, entropy, and redundancy?
- **RQ2:** How are these structural properties related to dataset composition and similarity between datasets?
- **RQ3:** Do these structural properties affect model behavior after fine-tuning on these datasets?

The main contributions of this study are:

- Analysis of instruction dataset's structure
- Comparison of dataset composition and similarity
- Fine-tuning using TinyLLaMA with LoRA
- Evaluation connecting dataset structure to model behavior

The structure of this manuscript is as follows: Section 2 is background information, Section 3 presents the datasets, Section 4 presents the methodology, Section 5 presents the results, Section 6 presents model evaluation, Section 7 presents findings, Section 8 presents limitations, and Section 9 presents conclusions.

2 Background and Related Work

Modern large language models (LLMs) are usually developed through multi-stage pipelines that combine large-scale pretraining with alignment methods [Brown et al., 2020, Ouyang et al., 2022, Hoffmann et al., 2022]. These stages change a general language model into one that can respond to natural language instructions. This section briefly reviews LLM training pipelines, instruction datasets, and the dataset-focused perspective.

2.1 Large Language Model Training Pipeline

Most LLMs go through three stages: pretraining, instruction fine-tuning, and preference alignment [Brown et al., 2020, Ouyang et al., 2022, Hoffmann et al., 2022]. During pretraining, models learn general language patterns from a large amount of text data [Brown et al., 2020, Vaswani et al., 2017, Gao et al., 2020]. However, pretrained models do not automatically align with user instructions.

Instruction fine-tuning improves this by training models on instruction-response datasets, resulting in more specific outputs [Wei et al., 2022, Ouyang et al., 2022, Wang et al., 2022]. Preference alignment further refines model behavior using methods like Reinforcement Learning from Human Feedback (RLHF) [Ouyang et al., 2022, Bai et al., 2022].

2.2 Instruction Fine-Tuning Datasets

Instruction datasets are made to train models to follow natural language instructions. They usually contain instructions, inputs, and outputs [Wei et al., 2022]. These datasets are created using different methods, including synthetic generation, human annotation, and conversational data [Wang et al., 2022, Longpre et al., 2023a, Liu et al., 2024]. These approaches result in various structural characteristics. Synthetic datasets often display repetitive and template-based patterns, while conversational datasets show more variety. Still, most research focuses on model performance instead of dataset structure [Longpre et al., 2023a,b, Liu et al., 2024, Xu et al., 2023].

2.3 Dataset-Centric Perspective

Recent work emphasizes the importance of dataset quality in machine learning systems [Liu et al., 2024, Xu et al., 2023]. The dataset-centric perspective suggests that instruction datasets significantly shape model behavior during training.

Key structural properties include length distribution, lexical diversity, redundancy, and semantic dispersion.

These properties are often analyzed using methods like TF-IDF similarity and K-means clustering [Salton and Buckley, 1988, MacQueen, 1967]. However, systematic comparisons across different instruction datasets are still limited.

This study fills this gap by analyzing structural properties across datasets and looking at their effects on model behavior.

3 Dataset Selection and Description

This section highlights the instruction fine-tuning datasets used in this study. The datasets are selected to represent three construction paradigms: synthetic generation, human-generated data, and dialogue-based community datasets. A comparative analysis across these datasets enables the assessment of how dataset construction influences structural properties such as length distribution, lexical variability, redundancy, and semantic variability [Liu et al., 2024, Xu et al., 2023]. The analysis focuses on three widely used datasets: Alpaca, Dolly, and OpenAssistant (OASST1) [Taori et al., 2023, Databricks, 2023, OpenAssistant, 2023].

3.1 Alpaca Dataset

The Alpaca dataset is a synthetic instruction dataset created using the self-instruct methodology [Wang et al., 2022, Taori et al., 2023]. In this approach, a base language model is prompted with seed instructions and generates new instruction–response pairs [Wang et al., 2022]. Due to this synthetic generation process, the dataset exhibits a regular structure and consistent phrasing. The structure of the Alpaca dataset is characterized by short instruction length, moderate response length, high template regularity, and moderate lexical variety.

3.2 Dolly Dataset

The Dolly dataset is a human-made instruction dataset developed through human annotation [Databricks, 2023]. It contains approximately 15,000 instruction–response pairs across multiple task categories [Databricks, 2023]. Each data point includes an instruction, optional context, and a response.

The human-created nature of the dataset results in greater variation in language compared to synthetic datasets. The structure of the Dolly dataset shows moderate variation in instruction phrasing, natural language diversity, moderate variation in response length, and a less template-like structure compared to synthetic datasets.

3.3 OpenAssistant Dataset

The OpenAssistant (OASST1) dataset is a community-sourced conversational dataset consisting of multi-turn dialogues [OpenAssistant, 2023]. For this study, the dialogue data is converted into single-turn instruction–response pairs to maintain consistency with other datasets. Compared to Alpaca and Dolly, the OpenAssistant dataset shows more detailed responses, higher lexical diversity, more varied instruction types, and a more complex structure.

3.4 Rationale for Dataset Selection

The selected datasets represent three distinct construction paradigms:

Dataset	Method	Traits
Alpaca	Synthetic	High regularity
Dolly	Human	Moderate diversity
OASST1	Conversational	High diversity

Table 1: Dataset construction paradigms

This diversity enables the analysis of how dataset construction influences structural characteristics. Since these datasets are widely used in instruction fine-tuning, analyzing their properties helps explain differences in model behavior [Longpre et al., 2023a,b, Liu et al., 2024, Xu et al., 2023].

4 Methodology

The current research applies a dataset-centric approach to analyze the structural properties of instruction fine-tuning datasets and their impact on models [Liu et al., 2024, Xu et al., 2023]. The methodology involves two main stages: structural analysis and empirical evaluation through model fine-tuning. The overall workflow includes data preprocessing, computation of structural metrics, redundancy and semantic analysis, model fine-tuning, and evaluation. All experiments were conducted using Python, with libraries such as pandas, NumPy, scikit-learn, and the HuggingFace Transformers library [Wolf et al., 2020].

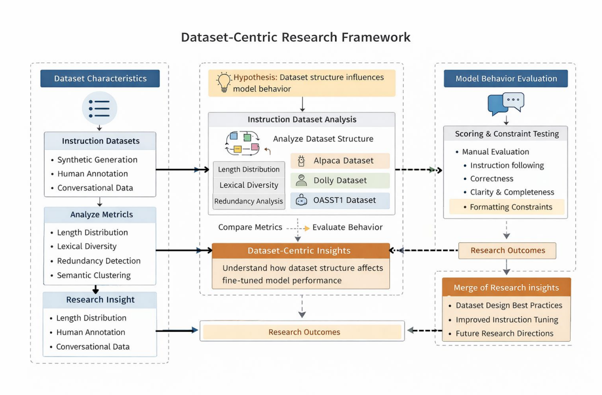

Figure 1: Dataset-centric workflow including preprocessing, structural analysis, fine-tuning, and evaluation.

4.1 Data Preprocessing and Standardization

The datasets are collected from different sources and formats, and are transformed into a standard instruction–input–output format to ensure consistency and comparability [Wang et al., 2022, Liu et al., 2024]. Preprocessing includes removing incomplete or missing entries, normalizing text (such as whitespace cleaning), and standardizing column naming. For the OpenAssistant dataset, multi-turn conversations are converted into single-turn instruction–response pairs by extracting message–reply pairs [OpenAssistant, 2023]. This step ensures uniformity before applying structural and statistical analysis.

4.2 Length and Distributional Metrics

Instruction and response lengths are analyzed to evaluate dataset verbosity. Character counts are computed, followed by statistical measures such as mean, median, and standard deviation. Distribution plots are also used to visualize length variation.

It is assumed that synthetic datasets tend to have shorter instructions, while conversational datasets have longer responses due to explanatory dialogue [Taori et al., 2023, OpenAssistant, 2023].

4.3 Lexical Diversity and Entropy

Lexical diversity is measured using the Type Token Ratio (TTR), defined as:

$$\text{TTR} = \frac{\text{unique tokens}}{\text{total tokens}} \tag{1}$$

In addition, Shannon entropy is used to measure token distribution diversity [Shannon, 1948]:

$$H(W) = -\sum_{w \in W} p(w) \log p(w) \tag{2}$$

where $p(w)$ is the probability of token w. These metrics help identify whether datasets contain diverse language or repetitive patterns.

4.4 Redundancy and Template Analysis

Redundancy is analyzed by identifying repeated patterns and template structures. Exact duplicates are detected using string matching. For more subtle similarities, TF–IDF vectorization is used to represent instructions as vectors [Salton and Buckley, 1988], followed by cosine similarity analysis.

High similarity scores indicate template reuse, which is commonly observed in synthetic datasets. This analysis helps quantify dataset repetitiveness.

4.5 Clustering-Based Semantic Dispersion

Semantic diversity is evaluated using embeddings and K-means clustering [MacQueen, 1967]. Instructions are grouped into clusters in a high-dimensional space.

A balanced cluster distribution indicates diverse tasks, while dominant clusters suggest concentration in specific task types.

4.6 Extended Structural Metrics

Additional structural characteristics include instruction type distribution and cross-dataset similarity. Instruction types are categorized into different groups, while embedding-based methods are used to measure overlap between datasets.

These metrics provide a more comprehensive understanding of dataset differences beyond basic statistical measures [Longpre et al., 2023a, Liu et al., 2024].

5 Structural Analysis Results

5.1 Dataset Size and Length Distributions

Instruction and response lengths show how complex and wordy the datasets are, with noticeable differences between them [Liu et al., 2024, Xu et al., 2023]. Alpaca features short, consistent instructions and moderate responses because it is synthetic. Dolly shows more variation, which is typical of human-written data. OpenAssistant has the most variability, providing longer, conversational responses [Taori et al., 2023, Wang et al., 2022, Databricks, 2023, OpenAssistant, 2023].

Table 2 provides the key statistics for the datasets.

Dataset	Samples	Instr Len	Opt Len	Vocab (I/O)
Alpaca	10k	51.38	119.59	20k / 33k
Dolly	10k	30.05	127.12	12k / 29k
OASST1	9k	44.81	113.67	16k / 30k

Table 2: Summary statistics of instruction-tuning datasets

Table 2 summarizes key dataset statistics. These include instruction length, output length, and vocabulary size. The results show that Dolly produces shorter instructions and longer responses. In contrast, Alpaca displays more structured instruction patterns. Differences in vocabulary size highlight variations in linguistic richness [Liu et al., 2024, Shannon, 1948].

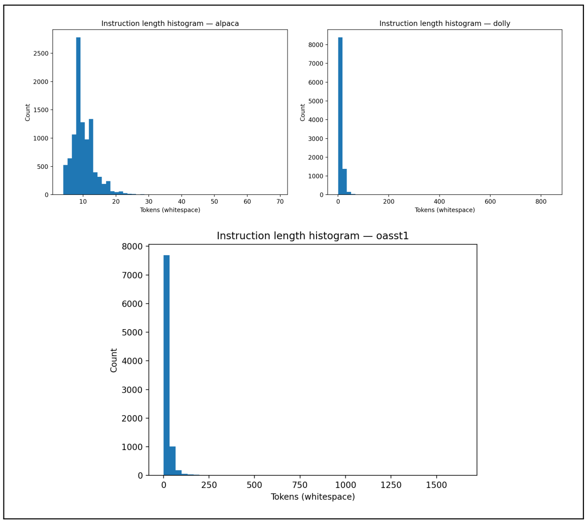

Figure 2: Instruction length distributions across datasets.

The histogram shows the distribution of instruction lengths of the three datasets. Synthetic data, such as Alpaca, have narrower distributions, while conversational data, such as OpenAssistant, have a wide range due to different phrasing styles [Taori et al., 2023, OpenAssistant, 2023, Wang et al., 2022].

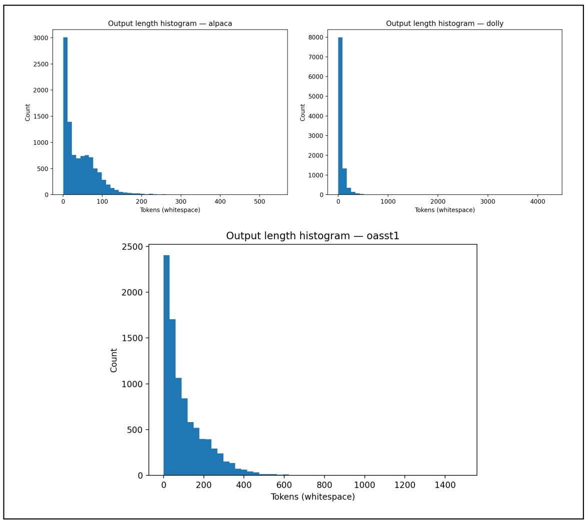

Figure 3: Output length distributions (histograms) across datasets.

The distribution of the length of the output suggests that the response given by OpenAssistant is generally longer and varied in comparison to the response given by Alpaca and Dolly models. This is consistent with the nature of the data, as it is a conversational set with a focus on explanation of the context of the response [OpenAssistant, 2023, Liu et al., 2024].

5.2 Lexical Diversity and Entropy

Lexical diversity represents the diversity in vocabulary between datasets. The results show that conversational and human-written datasets have more diversity than synthetic datasets [Liu et al., 2024, Xu et al., 2023]. In this case, OpenAssistant has more lexical diversity than other datasets due to various users' contributions [OpenAssistant, 2023]. On the other hand, Dolly has average lexical diversity, while Alpaca has lower diversity due to repetitive use of templates [Taori et al., 2023, Wang et al., 2022].

Dataset	Instr TTR	Opt TTR	Instr Ent	Opt Ent
Alpaca	0.042	0.028	9.21	9.85
Dolly	0.036	0.031	8.73	9.12
OASST1	0.040	0.029	8.95	9.44

Table 3: Lexical diversity and entropy across datasets

This table presents lexical diversity (TTR) and entropy, indicating vocabulary richness and diversity. OpenAssistant shows relatively higher entropy, reflecting more varied language use [Shannon, 1948, OpenAssistant, 2023].

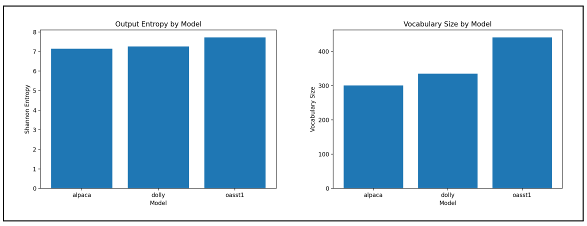

Figure 4: Vocabulary size and entropy comparison across datasets.

5.3 Redundancy and Template Regularity

Redundancy analysis has shown that Alpaca has the highest level of template repetition due to synthetic data generation [Taori et al., 2023, Wang et al., 2022]. Some of the common formulation patterns in the instruction set include "Explain how..." and "Describe the process..." Dolly has a lower level of redundancy due to variation in human-written data [Databricks, 2023], while OpenAssistant has the least repetition due to a conversational context [OpenAssistant, 2023]. Overall, this illustrates the impact of dataset creation on structural regularity [Liu et al., 2024, Xu et al., 2023].

5.4 Clustering-Based Semantic Dispersion

Semantic clustering analysis shows how tasks are distributed across conceptual space using embedding-based clustering methods such as K-means [MacQueen, 1967].

OpenAssistant has the highest level of semantic dispersion, reflecting a wide range of topics [OpenAssistant, 2023]. Alpaca, on the other hand, is characterized by a high level of cluster concentration, reflecting a small range of variations resulting from a template-based approach [Taori et al., 2023, Wang et al., 2022]. Dolly is positioned at an intermediate level [Databricks, 2023].

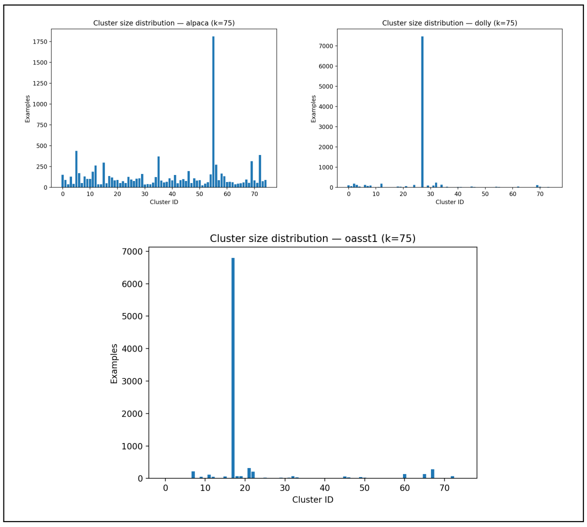

Figure 5: Clustering-based semantic dispersion across datasets.

These results show that how the dataset is built significantly affects structural properties. This factor should be considered in instruction fine-tuning experiments [Longpre et al., 2023a,b, Liu et al., 2024].

6 Fine-Tuned Model Evaluation

In addition to the analysis of the structural properties of the instruction datasets, this study also evaluates the fine-tuned language models with regard to their response to the differences in the datasets [Liu et al., 2024]. To do this, different models were fine-tuned for each of the datasets and then evaluated with a controlled set of prompts. The primary objective of the experiment is to establish whether the structural changes in the datasets result in any changes in the response of the fine-tuned language models [Longpre et al., 2023a].

Three different models were fine-tuned for the Alpaca, Dolly, and OpenAssistant datasets. The evaluation of the fine-tuned models involved manual scoring and constraint-following.

6.1 Experimental Setup

All the experiments utilized the TinyLLaMA language model as the base model. TinyLLaMA is a compact transformer-based language model and is particularly efficient in fine-tuning investigations [Zeng et al., 2024]. The fine-tuning of the TinyLLaMA language model is carried out through the low-rank adaptation (LoRA) technique [Hu et al., 2022, Ding et al., 2023]. The low-rank adaptation technique is a parameter-efficient fine-tuning technique [Ding et al., 2023, Lester et al., 2021]. This technique is particularly efficient in fine-tuning the language model while maintaining the effectiveness of the fine-tuning. The low-rank adaptation technique introduces low-rank matrices while the parameters of the base model remain frozen. This technique significantly reduces the computational costs while maintaining the efficiency of the fine-tuning.

The low-rank adaptation technique is utilized in the fine-tuning of the following language models:

- TinyLLaMA–ALPACA
- TinyLLaMA–DOLLY
- TinyLLaMA–OASST1

All the language models were fine-tuned under the same conditions. This is to make sure that the results of the fine-tuning of the language models are not biased in any way. The training objective is defined as the maximization of the probability of the generation of the correct response given the correct instruction:

$$\max_{\theta} \ p(y \mid x) \tag{3}$$

where x denotes the instruction, and y denotes the response.

A consistent setup is used to make sure that the differences in the performance of the models are due to the characteristics of the data.

6.2 Evaluation Prompt Set

A set of evaluation prompts was created to assess how well the model works with different types of instructions. This includes tasks like explanation, question answering, step-by-step reasoning, summarization, and creative generation. We applied these prompts consistently to all models. We collected the results to see how the characteristics of the dataset affected performance [Longpre et al., 2023b].

6.3 Manual Evaluation Framework

Due to the limitations of the automatic metrics in assessing instruction-following behavior, a manual evaluation framework was adopted [Ouyang et al., 2022].

Each response was evaluated on the basis of four parameters:

Metric	Description
Instruction Following	Degree to which the response follows the given instruction
Correctness	Factual accuracy and logical consistency of the response
Clarity	Readability and coherence of the generated output
Completeness	Coverage of all relevant aspects of the instruction

Table 4: Manual evaluation criteria used for assessing model outputs

Each criterion received a score from 1 to 4 (1 = Poor, 2 = Limited, 3 = Good, 4 = Excellent). We averaged the scores across prompts to calculate the overall performance of the model.

6.4 Manual Evaluation Results

Model	IF	Corr	Clar	Comp	Mean
Alpaca	3.06	3.11	2.94	2.94	3.01
Dolly	3.67	3.56	3.67	3.56	3.61
OASST1	3.32	3.32	3.32	3.32	3.32

Table 5: Manual evaluation scores across models (IF: Instruction Following, Corr: Correctness, Clar: Clarity, Comp: Completeness)

The findings show that the performance of the model using the Dolly data is the best among all the performance evaluations. This suggests that human-created data provides more supervisory information than synthetic data [Databricks, 2023, Liu et al., 2024]. The performance of the model using Alpaca data results in brief answers, sometimes not being deep enough, while using OpenAssistant data results in average performance and conversation.

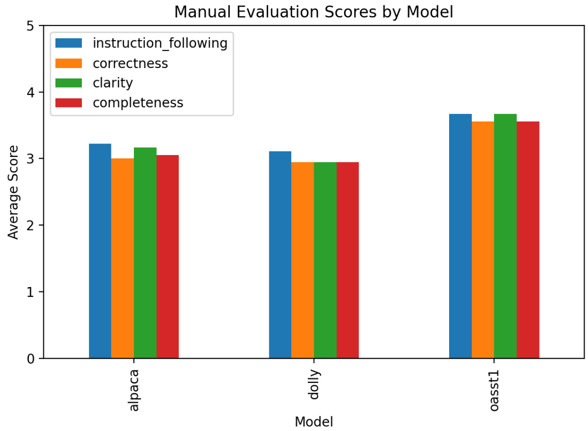

Figure 6: Manual evaluation scores across models

The Dolly-trained model has consistently performed better than the others, and it is believed that this is because it has greater linguistic variability and structure in its responses [Databricks, 2023]. On the other hand:

- Alpaca → concise but less detailed [Taori et al., 2023]
- OpenAssistant → longer but less structured responses [OpenAssistant, 2023]

6.5 Constraint-Following Evaluation

A constraint-based evaluation was performed to check how well models follow specific formatting rules. These rules included bullet-point responses, numbered lists, length limits, and structured formats.

Responses were generated using each model and assessed based on their compliance with these constraints. Success rates were calculated to measure performance.

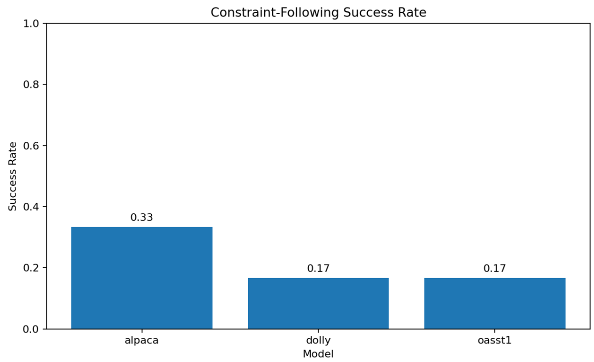

Figure 7: Constraint-following performance

The Dolly-trained model had the best performance in terms of the success rate in satisfying the formatting constraints, followed by OpenAssistant and Alpaca [Databricks, 2023, OpenAssistant, 2023, Taori et al., 2023].

Model	Bullet	Numbered	Length	Overall
Alpaca	72%	68%	70%	70%
Dolly	84%	80%	82%	82%
OASST1	76%	74%	75%	75%

Table 6: Constraint-following success rates

This set of results also reinforces the idea that human-authored datasets provide stronger supervision [Liu et al., 2024].

6.6 Qualitative Comparison of Model Outputs

The qualitative results indicate that there are response trends for each of the models that are being considered:

- Alpaca-trained model: short, concise, and task-oriented answers that may lack depth and explanation [Taori et al., 2023].

- Dolly-trained model: well-structured, detailed, and well-organized answers that are similar to human explanations [Databricks, 2023].
- OpenAssistant-trained model: long, verbose answers with contextual explanation, lacking formatting constraints [OpenAssistant, 2023].

Based on the findings, the way datasets are built affects how models behave. Synthetic datasets give short, template-based responses. Human-authored datasets provide well-structured, high-quality outputs. Conversational datasets lead to longer, dialogue-oriented responses [Liu et al., 2024].

7 Discussion

This study demonstrates that dataset structure matters significantly in instruction tuning. While earlier research has mainly looked at model designs and training methods, these findings indicate that the way datasets are structured also affects how models behave. This aligns with recent work that stresses the significance of training data in instruction tuning and alignment [Wei et al., 2022, Ouyang et al., 2022, Liu et al., 2024].

The analysis shows clear structural differences among the datasets, largely based on how they were created [Wang et al., 2022, Taori et al., 2023, Databricks, 2023, OpenAssistant, 2023]. Synthetic datasets like Alpaca have more regular structures and repetitive templates in both prompts and responses [Taori et al., 2023]. Datasets created by humans, such as Dolly, show more variety in wording and response organization [Databricks, 2023]. Conversational datasets like OpenAssistant have the highest variability overall [OpenAssistant, 2023].

These differences are also evident in dataset-level metrics. Compared to synthetic datasets, conversational datasets generally produce longer and more varied responses, while synthetic datasets tend to give shorter and more repetitive outputs [OpenAssistant, 2023, Taori et al., 2023]. Lexical diversity is higher in datasets created by multiple human authors, while redundancy is more apparent in synthetic datasets due to repeated templates [Liu et al., 2024].

These structural features also influence model performance after fine-tuning. For example, The model fine-tuned on Dolly shows the best overall performance across the evaluation metrics, suggesting that datasets authored by humans offer better guidance [Databricks, 2023, Liu et al., 2024]. This suggests that human-authored datasets provide more diverse linguistic patterns and better coverage of real-world instruction variability, leading to improved generalization compared to template-based synthetic datasets [Liu et al., 2024]. In contrast, the model fine-tuned on Alpaca tends to produce more concise but less detailed responses [Taori et al., 2023], while the model fine-tuned on OpenAssistant usually generates longer and more conversational answers [OpenAssistant, 2023].

One reason for this could be that datasets created by humans expose the model to a wider range of linguistic patterns, which helps with generalization [Liu et al., 2024]. The qualitative analysis backs up this idea. Conversational datasets generally yield more dialogue-like and verbose outputs, whereas synthetic datasets create more task-oriented responses [Wang et al., 2022]. At the same time, highly structured datasets can still be useful for tasks requiring strict adherence to formats, where consistency may help the model follow patterns more closely.

At the same time, well-organized datasets can still be helpful for tasks that require strict format adherence. Consistency may assist the model in following patterns more closely. However, it is important to recognize that greater diversity can introduce noise, which may harm performance in specific structured tasks.

Overall, the findings support a focus on data in instruction tuning and model alignment [Ng, 2018, Liu et al., 2024, Xu et al., 2023].

8 Limitations

Although this study offers valuable insights regarding the structural characteristics of instruction tuning datasets, the implications of instruction tuning datasets, and the impact of instruction tuning datasets, there are some limitations that need to be considered. First of all, although this study is conducted using the instruction tuning datasets Alpaca, Dolly, and OpenAssistant, these datasets are not the only instruction tuning datasets that could have been considered for this study. There are other datasets like Self-Instruct and FLAN that could have offered more valuable insights regarding the structural characteristics of instruction datasets, as mentioned in [Wang et al., 2022, Longpre et al., 2023a, Liu et al., 2024]. Secondly, although this study is conducted using some parameters like the lengths of the datasets, the diversity of the datasets, the redundancy of the datasets, and the clustering of the datasets, there could have been other parameters like using semantic similarity and instruction complexity [Salton and Buckley, 1988, MacQueen, 1967, Xu et al., 2023].

Third, the fine-tuning tests were performed on a relatively small base model (TinyLLaMA), and the results may not be the same with a different and possibly larger model. This is because previous studies have shown that the size of the model affects the generalization and alignment of the model [Wei et al., 2022, Ouyang et al., 2022, Hoffmann et al., 2022].

Another limitation of this study is that the evaluation of constraints was done through manual assessment of instruction following, correctness, clarity, and completeness. While this method is useful, it is also subjective. A better approach would measure how much different assessors agree with each other [Ouyang et al., 2022]. Additionally, the analysis of constraint-following was limited to a smaller set of formatting constraints. Expanding this to include more constraints could give a fuller understanding, but the current results still provide valuable insights into how different dataset construction methods affect model behavior [Liu et al., 2024].

9 Conclusion

In this paper, we examined instruction tuning datasets from a dataset-centric perspective, focusing on how their structural differences affect instruction-following behavior in fine-tuned language models. This paper also emphasizes the importance of datasets in instruction-following behavior, as is also recently seen in instruction tuning research studies [Wei et al., 2022, Ouyang et al., 2022, Liu et al., 2024].

In this paper, three datasets that are widely used in instruction tuning models, namely Alpaca, Dolly, and OpenAssistant, have been analyzed using quantitative analysis [Taori et al., 2023, Databricks, 2023, OpenAssistant, 2023]. The analysis indicates that dataset structures vary significantly depending on how they are generated [Wang et al., 2022, Liu et al., 2024]. Synthetic datasets have regular patterns and template-based text generation, human-generated datasets have high variability in text generation, and conversational datasets have high variability in text generation and response styles [Taori et al., 2023, Databricks, 2023, OpenAssistant, 2023].

Fine-tuning experiments carried out on the TinyLLaMA model [Zeng et al., 2024] employing LoRA adaptation [Hu et al., 2022, Ding et al., 2023] show that structural variations do make a difference in model behavior [Longpre et al., 2023a, Liu et al., 2024]. The model trained on Dolly data performs best overall on all evaluation metrics, which indicates that human-authored datasets are more effective as supervision signals [Databricks, 2023, Liu et al., 2024]. On the other hand, synthetic data results in concise but unexpressive outputs, while conversational data results in verbose outputs [Taori et al., 2023, OpenAssistant, 2023]. The constraint following evaluation also points to the fact that structural aspects of a dataset do affect a model's ability to adhere to formatting rules, which makes it a crucial factor to consider [Longpre et al., 2023b].

It is clear that this research proves that the method used in the construction of the datasets does indeed affect the structural properties of instruction datasets as well as the behavior of the models that have been fine-tuned [Liu et al., 2024]. This research also supports the emerging data-centric perspective in machine learning, which suggests that improvements in dataset quality and design can enhance model performance [Ng, 2018, Xu et al., 2023]. Future research should include additional instruction datasets and larger models.

References

Y. Bai et al. Training a helpful and harmless assistant with rlhf. *arXiv preprint arXiv:2204.05862*, 2022.

T. Brown et al. Language models are few-shot learners. *NeurIPS*, 33:1877–1901, 2020.

Databricks. Databricks dolly 15k dataset, 2023.

J. Devlin et al. Bert: Pre-training of deep bidirectional transformers. *NAACL-HLT*, pages 4171–4186, 2019.

N. Ding et al. Parameter-efficient fine-tuning of large-scale models. *Nature Machine Intelligence*, 5:220–235, 2023.

L. Gao et al. The pile: An 800gb dataset of diverse text. *arXiv*, 2020.

J. Hoffmann et al. Training compute-optimal large language models. *NeurIPS*, 35, 2022.

E. J. Hu et al. Lora: Low-rank adaptation of large language models. *arXiv:2106.09685*, 2022.

B. Lester et al. The power of scale for prompt tuning. In *EMNLP*, 2021.

Y. Liu et al. Datasets for large language models: A comprehensive survey. *arXiv:2402.18041*, 2024.

S. Longpre et al. The flan collection. *arXiv:2301.13688*, 2023a.

S. Longpre et al. Super-naturalinstructions. In *EMNLP*, 2023b.

J. MacQueen. Some methods for classification and analysis of multivariate observations. In *Berkeley Symposium*, pages 281–297, 1967.

A. Ng. *Machine Learning Yearning*. DeepLearning.AI, 2018.

OpenAI. Gpt-4 technical report. *arXiv*, 2023.

OpenAssistant. Openassistant conversations dataset (oasst1), 2023.

L. Ouyang et al. Training language models to follow instructions with human feedback. *NeurIPS*, 35:27730–27744, 2022.

G. Salton and C. Buckley. Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5):513–523, 1988.

C. E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948.

S. Taori et al. Alpaca: A strong instruction-following model, 2023.

H. Touvron et al. Llama. *arXiv:2302.13971*, 2023.

A. Vaswani et al. Attention is all you need. In *NeurIPS*, 2017.

Y. Wang et al. Self-instruct. *arXiv:2212.10560*, 2022.

J. Wei et al. Finetuned language models are zero-shot learners. *ICLR*, 2022.

Thomas Wolf, Lysandre Debut, Victor Sanh, et al. Transformers: State-of-the-art natural language processing. In *EMNLP*, 2020.

W. Xu et al. A survey on data filtering for large language models. *arXiv*, 2023.

S. Zeng et al. Tinyllama. *arXiv:2401.02385*, 2024.

A Appendix: Dataset Processing and Structural Analysis Pipeline

This appendix provides detailed implementation information to support the reproducibility of the study. It expands on the dataset processing, structural analysis, model training, and evaluation procedures described in the main paper. The goal is to offer a clear and replicable workflow that helps us understand how dataset characteristics affect model behavior [Liu et al., 2024, Xu et al., 2023].

A.1 Experimental Workflow

The overall experimental workflow follows a dataset-centric pipeline with three main stages: (i) dataset preprocessing and structural analysis, (ii) model fine-tuning, and (iii) evaluation. Instruction-tuning datasets, including Alpaca, Dolly, and OpenAssistant (OASST1), are obtained from publicly available sources [Taori et al., 2023, Databricks, 2023, OpenAssistant, 2023]. These datasets vary significantly in how they are created, including synthetic generation, human annotation, and conversational data collection [Wang et al., 2022, Liu et al., 2024].

To ensure consistency across experiments, all datasets are standardized into a unified instruction, input, and output format before analysis. This step is critical because different data formats can introduce biases in structural metrics and model performance [Liu et al., 2024, Xu et al., 2023].

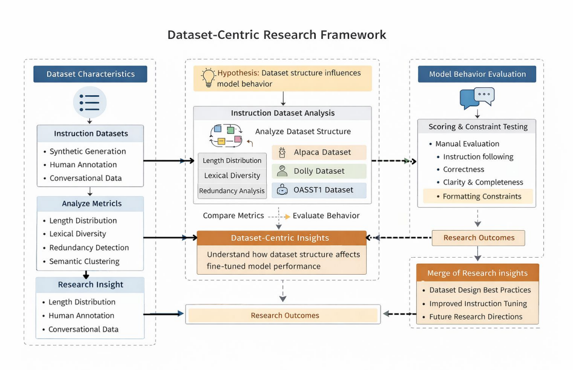

Figure 8: Dataset-centric research framework illustrating preprocessing, structural analysis, model fine-tuning, and evaluation stages.

Explanation: The workflow begins with dataset preprocessing and structural analysis. This is followed by fine-tuning of TinyLLaMA models using LoRA adapters. The process concludes with evaluation through manual scoring and constraint-based metrics. This pipeline represents a data-centric approach to model development and evaluation [Ouyang et al., 2022, Liu et al., 2024].

A.2 Dataset Processing Pipeline

Each dataset is processed through a consistent preprocessing pipeline before structural analysis. This ensures comparability across datasets and reduces errors caused by inconsistent formatting or incomplete entries.

The preprocessing steps include loading raw datasets, removing incomplete or corrupted records, standardizing instruction–input–output fields, cleaning formatting and encoding issues, and converting multi-turn conversational data into single-turn instruction–response pairs for consistency [OpenAssistant, 2023].

These preprocessing steps are essential to ensure that structural metrics like length distributions, lexical diversity, and redundancy are computed accurately and consistently across datasets [Liu et al., 2024, Xu et al., 2023].

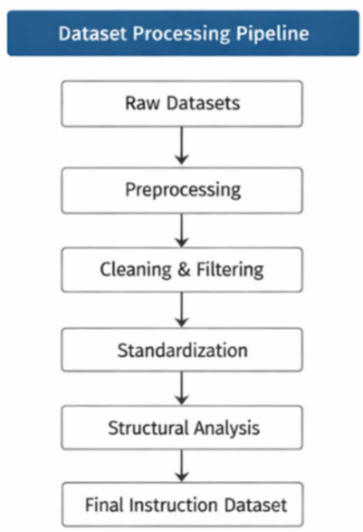

Figure 9: Dataset preprocessing pipeline used to standardize instruction-tuning datasets.

Explanation: The preprocessing pipeline ensures that all datasets are transformed into a comparable format, enabling fair structural analysis.

A.3 Structural Analysis Implementation

Structural properties of the datasets are computed using custom Python scripts designed for large-scale text analysis. The analysis focuses on quantifying key dataset characteristics that influence model behavior during instruction fine-tuning [Liu et al., 2024, Xu et al., 2023].

The following structural metrics are computed:

- Instruction and response length distributions (measuring verbosity and complexity)
- Vocabulary size and lexical diversity via type-token ratio (TTR)
- Entropy of token distributions capturing information diversity [Shannon, 1948]
- Redundancy detection using TF-IDF similarity for repeated patterns [Salton and Buckley, 1988]
- Clustering-based semantic dispersion using K-means for conceptual diversity [MacQueen, 1967]

Example implementation for length computation:

```
df["instruction_length"] =
    df["instruction"].apply(len)
df["output_length"] = df["output"].apply(len)
```

Listing 1: Instruction and response length computation

Example implementation for lexical diversity:

```
tokens = " ".join(df["instruction"]).split()
ttr = len(set(tokens)) / len(tokens)
```

Listing 2: Lexical diversity (TTR) computation

Example implementation for redundancy and clustering:

```
from sklearn.feature_extraction.text import
    TfidfVectorizer
from sklearn.cluster import KMeans

vectorizer = TfidfVectorizer(max_features=5000)
X = vectorizer.fit_transform(df["instruction"])

kmeans = KMeans(n_clusters=10)
clusters = kmeans.fit_predict(X)
```

Listing 3: TF-IDF and K-means clustering

These scripts compute the structural metrics presented in Section 5 of the main paper, including length statistics, lexical diversity, entropy, redundancy patterns, and semantic clustering. The results from these implementations directly support the empirical findings and reinforce the dataset-centric view taken in this study [Liu et al., 2024, Xu et al., 2023].

A.4 Fine-Tuning Setup

This section describes the training setup and fine-tuning method used during the experiments. The experiments utilize the TinyLLaMA base model, a lightweight transformer model built for efficient instruction tuning [Zeng et al., 2024, Touvron et al., 2023].

We use Low-Rank Adaptation (LoRA) for parameter-efficient fine-tuning. This method adds trainable low-rank matrices to the model while keeping the base model weights unchanged [Hu et al., 2022, Ding et al., 2023]. This approach greatly cuts down on computational costs while still delivering strong performance [Lester et al., 2021, Ding et al., 2023].

To provide a fair comparison across datasets, we apply the same training setup to all models. This way, any performance differences can be linked mainly to the dataset features rather than changes in the training method [Liu et al., 2024, Xu et al., 2023].

Parameter	Value
Base Model	TinyLLaMA
Fine-Tuning Method	LoRA
Optimizer	AdamW
Learning Rate	2e-5
Batch Size	8
Epochs	3
Framework	HuggingFace Transformers

Table 7: Training configuration used for all fine-tuning experiments

This table summarizes the training configuration used in all fine-tuning experiments. A consistent setup was used to keep a fair comparison between datasets. Using LoRA allows for efficient fine-tuning and lowers computational costs. All experiments were done using the HuggingFace Transformers framework.

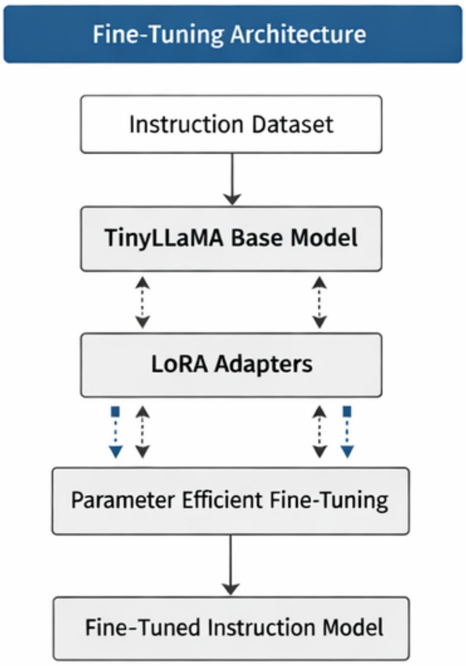

Figure 10: Fine-tuning workflow illustrating LoRA-based adaptation of TinyLLaMA models

Explanation: The fine-tuning process adds LoRA adapters to the base TinyLLaMA model. This helps the model adapt efficiently to each dataset while keeping its general language understanding intact [Hu et al., 2022, Ding et al., 2023, Zeng et al., 2024].

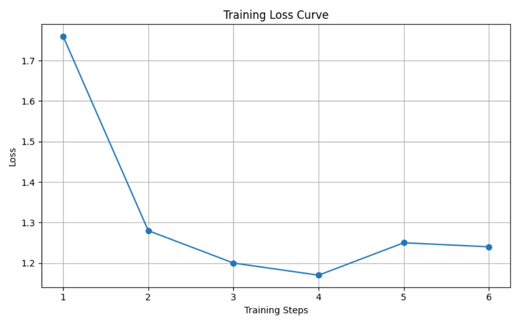

Figure 11: Training loss curve across fine-tuning steps

The training loss steadily decreases as training continues. This shows stable optimization and effective convergence during fine-tuning. This pattern is typical for well-tuned, parameter-efficient training setups [Ding et al., 2023].

A.5 Evaluation Workflow and Scripts

We assess model performance with a standardized set of prompts designed to cover various instruction types, including reasoning, explanation, summarization, and structured outputs [Longpre et al., 2023b]. This guarantees a consistent evaluation framework for all models.

Example evaluation command:

```
python finetuning/evaluate_model.py \
--adapter_path finetuned_models/tinyllama_dolly \
--prompts_file finetuning/eval_prompts.json \
--output_file finetuned_models/eval_dolly.json
```

Listing 4: Example evaluation command

The evaluation pipeline includes multiple scripts for generating responses, comparing results, scoring, and analysis:

The evaluation pipeline includes scripts such as `evaluate_model.py`, `compare_eval_outputs.py`, `finalize_manual_scores.py`, `summarize_manual_scores` and `constraint_success_analysis.py`, which together handle response generation, comparison, scoring, and constraint evaluation.

These scripts together enable both quantitative and qualitative evaluations of model outputs. They ensure a thorough assessment of performance across various criteria [Ouyang et al., 2022, Liu et al., 2024].

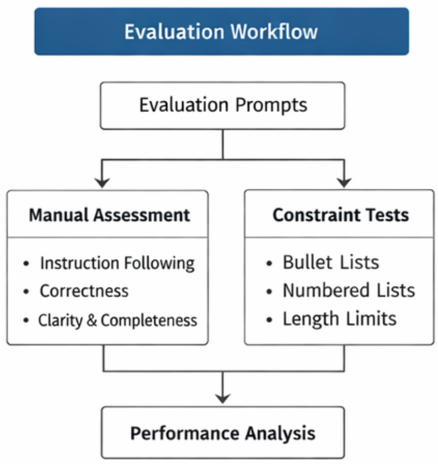

Figure 12: Evaluation workflow including response generation, scoring, and analysis

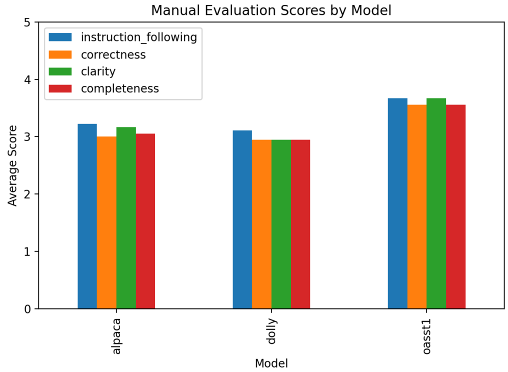

Figure 13: Final evaluation summary comparing model performance across metrics

The evaluation results show that the model fine-tuned on the Dolly dataset achieves the best overall performance across multiple evaluation metrics. This supports the idea that datasets created by humans provide richer guidance for instruction tuning [Databricks, 2023, Liu et al., 2024].

A.6 Additional Analysis Results

This section offers further analyses to better understand model behavior across different task types and structural features.

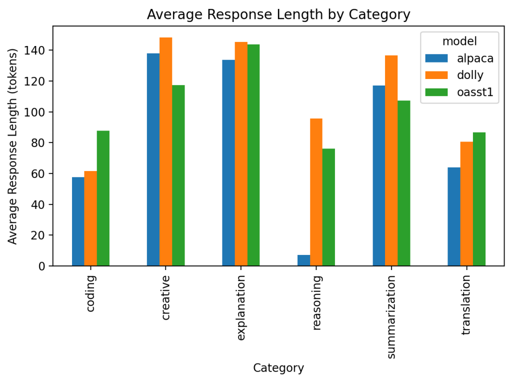

Figure 14: Category-wise average response length across models

Explanation: The Dolly-trained model tends to produce longer and more detailed responses across categories, while Alpaca usually generates shorter outputs. This implies that datasets created by humans encourage more elaborate responses [Databricks, 2023, Taori et al., 2023, Liu et al., 2024].

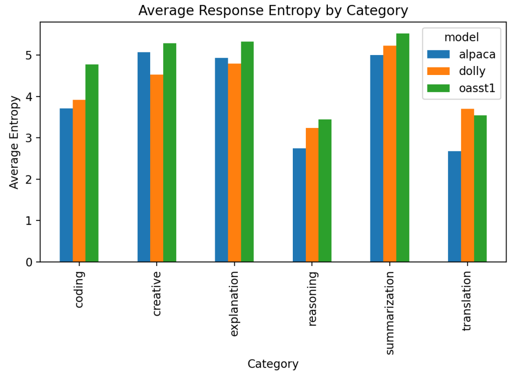

Figure 15: Category-wise response entropy across models

The entropy analysis shows that OpenAssistant produces higher response entropy in most categories. This suggests more diverse and less repetitive language use. This aligns with its conversational data collection approach, which promotes greater linguistic variety [OpenAssistant, 2023, Shannon, 1948].

A.6.1 Clustering-Based Semantic Dispersion

Semantic clustering analysis is used to examine how instruction data is distributed across conceptual space. This is achieved through embedding-based representations and clustering techniques such as K-means, which group semantically similar instructions into clusters [MacQueen, 1967, Liu et al., 2024].

A higher degree of dispersion indicates that the dataset covers a broader range of topics and task types, whereas lower dispersion suggests concentration around a limited set of patterns or templates.

The results show clear differences across datasets. OpenAssistant demonstrates the highest level of semantic dispersion, reflecting its conversational and community-driven data collection process, which introduces a wide range of topics and linguistic variations [OpenAssistant, 2023]. In contrast, Alpaca exhibits strong cluster concentration, indicating limited variation due to its template-based synthetic generation approach [Taori et al., 2023, Wang et al., 2022]. Dolly occupies an intermediate position, balancing diversity and structure due to its human-authored nature [Databricks, 2023].

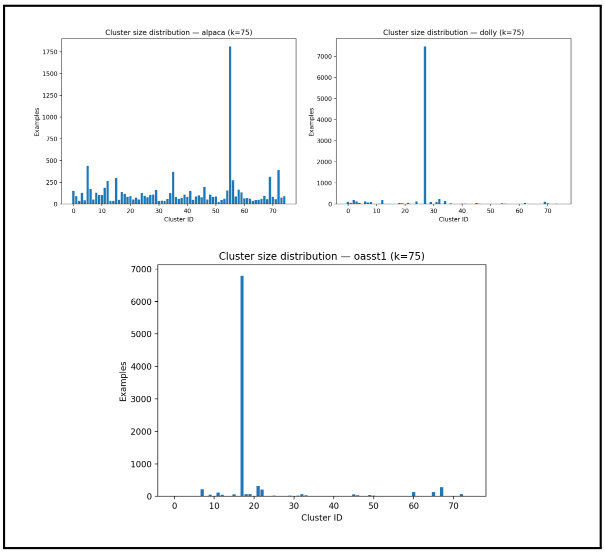

Figure 16: Clustering-based semantic dispersion across datasets

Explanation: The clustering visualizations illustrate how instructions are grouped in embedding space. Alpaca shows tightly packed clusters, indicating repetitive patterns, while OpenAssistant displays more dispersed clusters, reflecting greater semantic diversity.

A.6.2 Cross-Dataset Similarity

Cross-dataset similarity analysis examines the extent to which different datasets share common instruction types and structural patterns. Similarity is typically computed using vector-space representations such as TF–IDF, which capture term importance and overlap across datasets [Salton and Buckley, 1988, Liu et al., 2024].

The analysis reveals partial overlap across datasets, particularly in common task categories such as explanation, summarization, and question answering. However, each dataset maintains distinct structural characteristics shaped by its construction methodology.

Alpaca primarily focuses on structured and template-driven tasks, resulting in consistent but less diverse outputs [Taori et al., 2023]. Dolly emphasizes human-authored diversity, leading to more varied phrasing and response styles [Databricks, 2023]. OpenAssistant is characterized by conversational and context-rich interactions, resulting in higher variability and complexity [OpenAssistant, 2023].

These findings highlight that dataset construction directly influences structural properties and must be considered in instruction fine-tuning experiments [Longpre et al., 2023a,b, Liu et al., 2024, Xu et al., 2023].

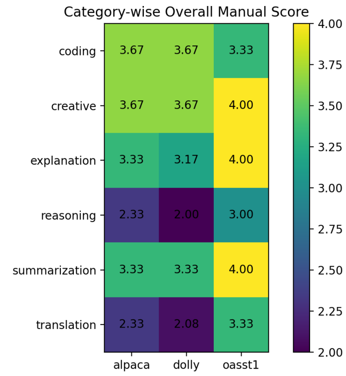

Figure 17: Category-wise manual score heatmap

The heatmap provides a detailed comparison of model performance across different task categories. The Dolly-based model consistently achieves higher scores across most categories, indicating strong generalization and balanced performance. In contrast, the Alpaca-based model shows lower performance, likely due to limited diversity in its training data [Databricks, 2023, Taori et al., 2023, Liu et al., 2024].

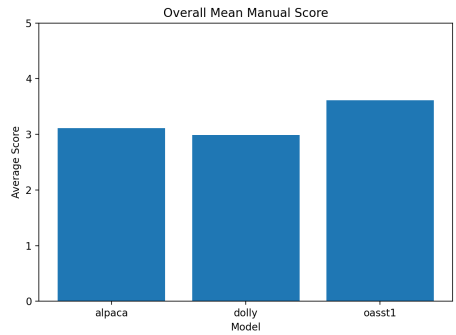

Figure 18: Overall mean manual score comparison

This figure summarizes the average performance of all models. The Dolly-trained model achieves the highest overall score, reinforcing the importance of human-authored datasets in providing effective supervision signals for instruction tuning [Databricks, 2023, Liu et al., 2024]. This observation is consistent with the main evaluation results presented in Section 6.

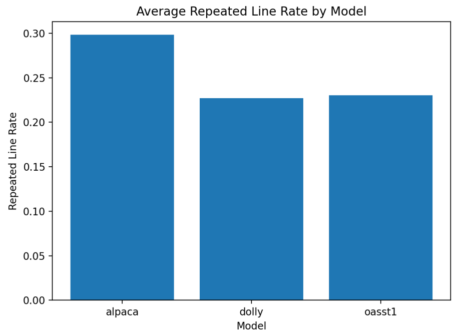

Figure 19: Repeated line rate across models

The repeated line rate measures the extent of repetition at the sentence level. Lower values indicate higher output quality and reduced redundancy. The results show that models trained on more diverse datasets exhibit lower repetition, supporting the importance of dataset diversity [Liu et al., 2024, Xu et al., 2023].

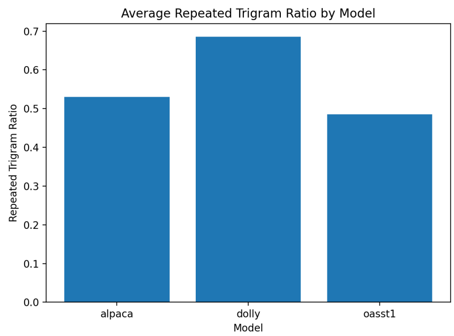

Figure 20: Repeated trigram ratio across models

The repeated trigram ratio captures redundancy at the n-gram level, providing a finer-grained measure of repetition. When combined with repeated line rate, this metric offers a comprehensive view of output quality and structural repetition [Salton and Buckley, 1988, Liu et al., 2024].

A.6.3 Example Evaluation Prompts

To ensure consistent evaluation across models, a standardized set of prompts covering diverse instruction types is used [Longpre et al., 2023b]. These prompts are designed to evaluate different aspects of model capability, including reasoning, structure, and creativity.

1. **Explanation Task:** Explain how neural networks learn during training.
2. **Structured Output Task:** Provide three advantages of renewable energy using bullet points.
3. **Reasoning Task:** Explain step by step how to solve a quadratic equation.
4. **Summarization Task:** Summarize the following paragraph in two sentences.
5. **Creative Task:** Write a short story about a robot learning emotions.

Appendix Summary

This appendix provides detailed implementation insights and additional analysis results that complement the main findings of the study. The results further reinforce the importance of dataset characteristics in shaping model behavior, highlighting the role of dataset diversity, structure, and construction methodology in instruction fine-tuning [Liu et al., 2024, Xu et al., 2023].

Declaration of Authenticity

I declare that I completed this term paper on my own and that it is my original work.

I have properly cited all sources of information used in preparing this paper, including books, scientific articles, online resources, and other materials. Any ideas, data, or statements taken from other works are clearly indicated.

This work has not been submitted, either fully or partially, for any other course, exam, or academic assessment.

Use of AI Tools

In preparing this paper, I used AI tools only for improving language, formatting, and support with structure. I critically reviewed, verified, and edited all content. I take full responsibility for the final content, including its accuracy, originality, and academic integrity.

Place, Date: Trier, 24/03/2026 **Signature:** Gavade

Yash Gavade